

POKAZNA VEŽBA 9

LPRS1 Assembler

Potrebno predznanje

- Arhitektura LPRS1 procesora
- Assembler

Šta će biti naučeno tokom izrade vežbe?

Nakon urađene vežbe, bićete u mogućnosti da:

- Napišete programsku podršku za LPRS1 računarski sistem u assembleru
- Simulirate rešenje i proverite ispravnost u ModelSim alatu

Apstrakt i motivacija

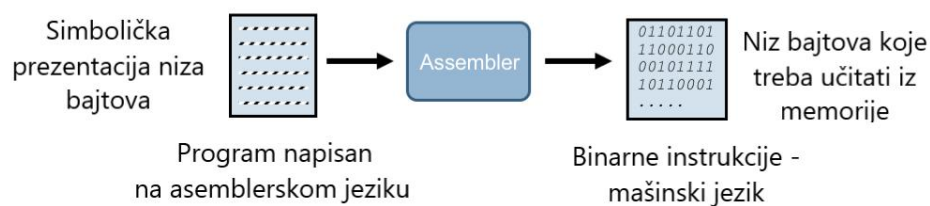
Kako bi izvršavali programe na LPRS1 procesoru, potrebno je zadati instrukcije u instrukcionoj memoriji. One moraju biti u odgovarajućem formatu kao niz nula i jedinica. Ovakvi nizovi nisu lako čitljivi korisniku i programeru i za svaku instrukciju je potrebno proveravati kôd operacije i operande na nivou nula i jedinica. Da ne bismo programirali procesor na ovom nivou, uvodi se asemblerski jezik koji omogućava jednostavno zadavanje operacije i operanada za svaku instrukciju. Dakle, umesto da pišemo binarni zapis svake instrukcije, pišaćemo izraze na asemblerskom jeziku kako bi simbolički predstavili željenu instrukciju.

TEORIJSKE OSNOVE

1. LPRS1 assembler i asemblerski jezik

Da bismo programirali LPRS1 procesor, tačnije na procesoru izvršavali neki program, potrebno je da učitavamo i izvršavamo 15-bitne binarne instrukcije iz memorije za instrukcije. Razumevanje svake 15-bitne instrukcije koja je napisana kao niz nula i jedinica je jednostavan posao za računar tj. procesor, ali nije jednostavno pisati složene programe na nivou binarnih instrukcija. Zbog toga je napravljen jednostavan programski jezik – **assemblerski jezik**, koji će nam omogućiti da zadajemo instrukcije u jednostavnom i čitljivom formatu. Dakle, umesto da pišemo binarni zapis svake instrukcije, napisaćemo izraze na asemblerskom jeziku kako bismo simbolički predstavili željenu instrukciju. Naravno da i dalje moramo voditi računa koje registre koristimo i paziti da se neke važne vrednosti u registrima ne prepisu korišćenjem narednih instrukcija.

U ovoj vežbi ćemo naučiti LPRS1 asemblerski jezik koji ćemo koristiti za pisanje programa i programiranje LPRS1 procesora. Prevodilac **ASSEMBLER** čita tekstualnu datoteku koja sadrži kôd napisan na **assemblerskom jeziku** i prevodi zadati program u **niz instrukcija** koje će se izvršavati na procesoru. Na Slici 1-1 je prikazan Assembler kao prevodilac programa, koji prevodi kôd napisan na **assemblerskom jeziku** u niz binarnih instrukcija (mašinski jezik).



Slika 1- 1. Assembler

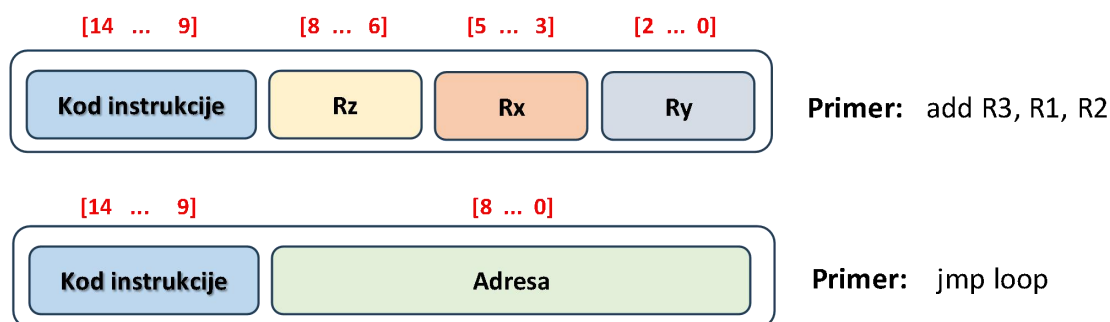
Da bismo napisali program u asemblerskom jeziku, moramo poznavati arhitekturu procesora, tj. koje operacije može da izvršava, instrukcijski set, kao i formate instrukcija koje mogu da se izvrše na željenom procesoru. Podsetimo se da LPRS1 procesor podržava tri vrste instrukcija:

1. **Instrukcije za računanje** – aritmetičke i logičke operacije nad vrednostima registara
2. **Instrukcije za učitavanje i smeštanje podataka** – pristupaju podacima u memoriji za podatke
3. **Instrukcije skoka/grananja** – menjaju vrednost programskog brojača.

Svaka instrukcija je kodovana sa **15 bita**. Format instrukcije zavisi od tipa instrukcije i može biti u jednom od formata koji su prikazani na slici 1-2:

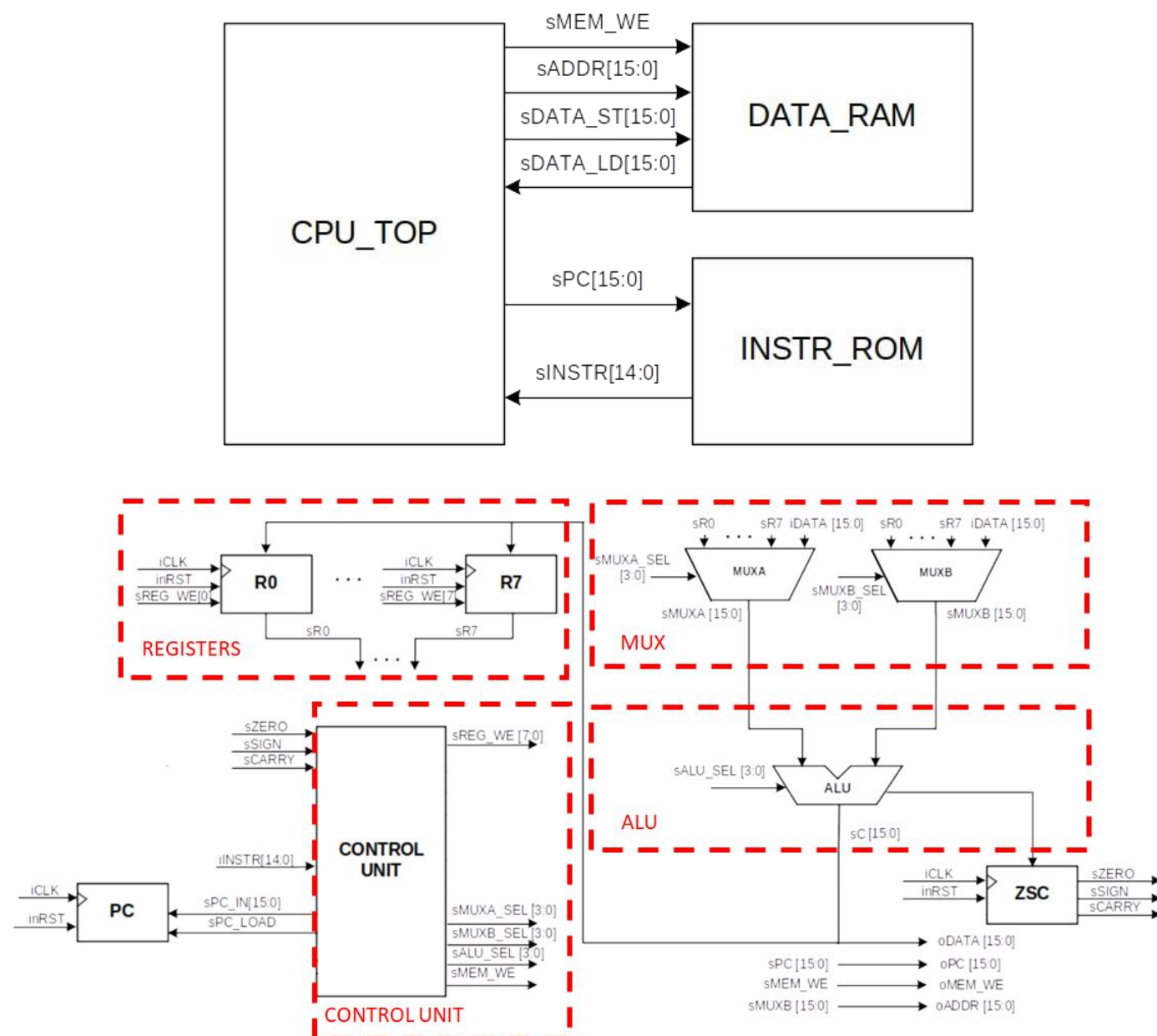
1. **IIIIIIZZZXXYYY**
2. **IIIIIIAAAAAAAAA**

Format instrukcije zavisi od toga da li instrukcija treba nešto da izračuna ili da menja adresu (skok u programu). Oba formata instrukcije sadrže na početku kod instrukcije širine 6 bita označen sa **IIIIII**. U prvom formatu je sa **ZZZ** označena adresa odredišnog registra **Rz** gde treba upisati rezultat, sa **XXX** adresa registra **Rx** (operand 1), a sa **YYY** adresa registra **Ry** (operand 2). Ukoliko se koristi drugi format instrukcije, 9-bitni deo **AAAAAAAAA** predstavlja adresu skoka. Kôd instrukcije označen sa **IIIIII** se može predstaviti sa **TTOOOO**, gde **TT** označava tip instrukcije, a **OOOO** označava kôd operacije, koji će dodatno biti opisan u opisu instrukcija.



Slika 1-2 – Formati instrukcije

Na slici 1-3 je prikazana arhitektura LPRS1 procesora, a u nastavku tabela 1-1 koja sadrži sve podržane instrukcije.



Slika 1-3. Arhitektura LPRS1 procesora

Operacija	Funkcija	Kôd instrukcije	Opis instrukcije
mov Rz, Rx	$RZ \leftarrow [RX]$	00 0000	Prebacuje sadržaj registra X u registar Z
add Rz, Rx, Ry	$RZ \leftarrow [RX] + [RY]$	00 0001	Zbir registara X i Y upisuje u registar Z
sub Rz, Rx, Ry	$RZ \leftarrow [RX] - [RY]$	00 0010	Razliku registara X i Y upisuje u registar Z
and Rz, Rx, Ry	$RZ \leftarrow [RX] \& [RY]$	00 0011	Rezultat logičkog I registara X i Y upisuje u registar Z
or Rz, Rx, Ry	$RZ \leftarrow [RX] \mid [RY]$	00 0100	Rezultat logičkog Ili registara X i Y upisuje u registar Z
not Rz, Rx	$RZ \leftarrow \text{not } [RX]$	00 0101	Negiranu vrednost registra X upisuje u registar Z
inc Rz, Rx	$RZ \leftarrow [RX] + 1$	00 0110	Inkrementira registar X i upisuje u registar Z
dec Rz, Rx	$RZ \leftarrow [RX] - 1$	00 0111	Dekrementira registar X i upisuje u registar Z
shl Rz, Rx	$RZ \leftarrow \text{shl } [RX]$	00 1000	Logički pomeren registar X ulevo upisuje u registar Z
shr Rz, Rx	$RZ \leftarrow \text{shr } [RX]$	00 1001	Logički pomeren registar X udesno upisuje u registar Z
ashl Rz, Rx	$RZ \leftarrow \text{ashl } [RX]$	00 1010	Aritmetički pomeren registar X ulevo upisuje u registar Z
ashr Rz, Rx	$RZ \leftarrow \text{ashr } [RX]$	00 1011	Aritmetički pomeren registar X udesno upisuje u reg Z
ld Rz, Ry	$RZ \leftarrow [[RY]]$	10 0000	Upisuje u Rz sadržaj koji se nalazi u memoriji na adresi u Ry
st Rx, Ry	$[[RY]] \leftarrow [RX]$	11 0000	Upisuje u memoriju na adresu koja piše u Ry sadržaj iz Rx
jmp ADDR	$PC \leq \text{ADDR}$	01 0000	Bezuslovni skok na labelu ADDR
jmpz ADDR	if Z = 1 jmp	01 0001	Uslovni skok na labelu ADDR ukoliko je rezultat 0
jmps ADDR	if S = 1 jmp	01 0010	Uslovni skok na labelu ADDR ukoliko je rezultat negativan
jmpc ADDR	if C = 1 jmp	01 0011	Uslovni skok na labelu ADDR ukoliko je došlo do prenosa
jmpnz ADDR	if Z != 1 jmp	01 0101	Uslovni skok na labelu ADDR ukoliko rezultat nije 0
jmpns ADDR	if S != 1 jmp	01 0110	Uslovni skok na labelu ADDR ukoliko rezultat nije negativan
jmpnc ADDR	if C != 1 jmp	01 0111	Uslovni skok na labelu ADDR ukoliko nije došlo do prenosa

2. Asemblerske direktive i konvencije

Assembler podržava sledeće konvencije za pisanje programa na asemblerskom jeziku:

- **Direktive** – označavaju početak sekcije i uvek počinju **tačkom**. Postoje dve vrste direktiva:
 - Direktiva za sekciju podataka **.data** – označava početak sekcije gde se nalaze podaci
 - Direktiva za tekst sekciju **.text** – označava početak sekcije gde se nalaze instrukcije
- **Simboli** nam omogućavaju da damo imena određenim vrednostima. Nazivi registara R0-R7 su definisani simboli za registre od 0 do 7.
- **Labele** predstavljaju simbole čije su vrednosti zapravo adrese određene lokacije u programu.
- **Makroi** predstavljaju skraćeni zapis za niz izraza, tj. instrukcija, koji će, kada se izvršavaju, generisati binarne reprezentacije za instrukcije i podatke. Kada assembler izvršava makro, on se zamenjuje sa njegovom definicijom i argumenti se postavljaju umesto odgovarajućeg teksta koji predstavlja makro definiciju.
- **Komentari** se pišu sa 2 kose linije „//“ nakon čega asemblerski prevodilac ignoriše sve simbole. Takođe se mogu koristiti i blok komentari **/* */** kao u programskom jeziku C.

NAPOMENA: Uobičajeno je da se piše jedan izraz po liniji i poželjno je koristiti prazna mesta, TAB-ove i nove linije kako bi kod bio čitljiviji. Takođe, poželjno je pisati i komentare u kodu.

U nastavku sledi jednostavan primer programa napisanog u LPRS1 asemblerskom jeziku. Program sabira 3 elementa niza iz memorije za podatke.

```

/*
    Ekvivalentan C kod:

    short niz[3] = {5, 3, 7};
*/
.data
    5          // niz[0], adresa 0
    3          // niz[1], adresa 1
    7          // niz[2], adresa 2
/*
    Ekvivalentan C kod:

    short tmp, acc = 0, i = 0, N = 3;
    do
    {
        tmp = niz[i];
        acc += tmp;
        niz[i] = acc;
        i++;
    } while(i != N);
*/
.text
/*
    Spisak registara:
    R0 - privremeni registar.
    R1 - dužina niza N
    R2 - indeks/adresa niza i
    R3 - akumulator (rezultat)
*/
begin:
    // N = 3; Realizacija u 3 instrukcije.
    inc R0, R0      // tmp++; Postavlja R0 na 1.
    shl R1, R0      // N = tmp << 1; Postavlja R1 na 2.
    add R1, R1, R0   // N = N + tmp; Postavlja R1 na 3.
loop:
    ld R0, R2       // tmp = niz[i]; R0 sadrži vrednost iz niza sa indeksa R2.
    add R3, R3, R0   // acc += tmp; Na R3 se nadodaje vrednost iz niza.
    st R3, R2       // niz[i] = acc; Upisuje se R3 u niz na indeks R2
    inc R2, R2      // i++; Uvećava se indeks R2.
    // } while(i != N); Realizacija u 2 instrukcije.
    sub R0, R2, R1   // i != N; Rezultat se odbacuje u R0.
    jmpnz loop      // do{ }while(); Ako su različiti, idi na početak petlje.
end:
    jmp end         // Beskonačna petlja. Rezultat je u R3.

```

3. Asembliranje i testiranje

Asembliranje je prevođenje programa napisanog u asemblerskom jeziku u mašinski format odnosno u odgovarajući format instrukcija koje će izvršiti procesor, u ovom slučaju LPRS1 procesor.

Sve programi napisani u assembleru moraju da se nalaze u direktorijumu **src/asm**.

Ukoliko je potrebno prevesti i izvršiti neki drugi program napisan u asemblerskom jeziku, potrebno je izmeniti skriptu za prevođenje koja se nalazi u glavnom direktorijumu projekta i naziva se **asm.tcl**. Dozvoljeno je menjati samo **naziv datoteke** koja sadrži program napisan u assembleru:

```

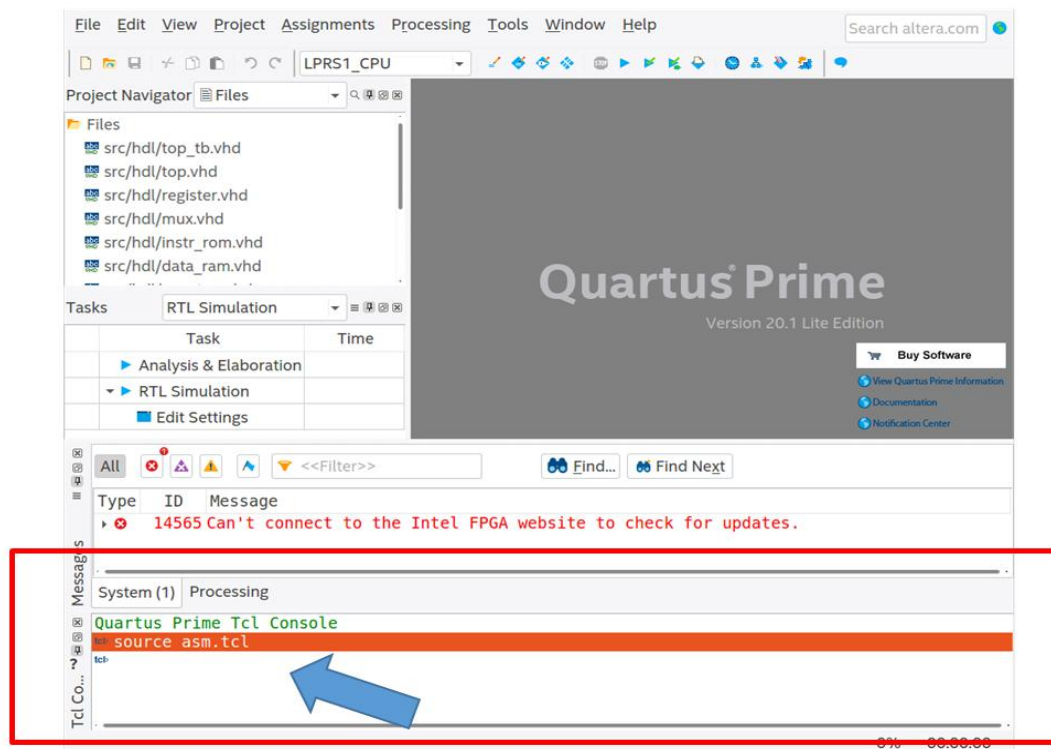
1  #!/usr/bin/env tcl
2  #####
3  # LPRS1 assembler script.
4  # Author: Milos Subotic <milos.subotic@uns.ac.rs>
5  #####
6
7  set asm_file "src/asm/cumsum.asm.txt"
8
9  #####
10
11 set OS [lindex $tcl_platform(os) 0]
12 set result [
13     catch {
14         if { $OS == "Windows" } {

```

Za pokretanja assemblera, otvoriti „Tcl Console“ korišćenjem prečice **Alt+2** i u konzoli upisati komandu praćenu enterom:

```
source asm.tcl
```

Slika 3-1 prikazuje izgled Quartus okruženja prilikom asembliranja.



Slika 3-1 - Asembliranje u Quartusu

Nakon asembliranja, promeniće se sadržaj sledećih vhdL datoteka: **data_ram.vhdl** i **instruction_rom.vhdl**. Ukoliko su ove datoteke bile otvorene, potrebno je prihvatiti izmene nakon asembliranja.

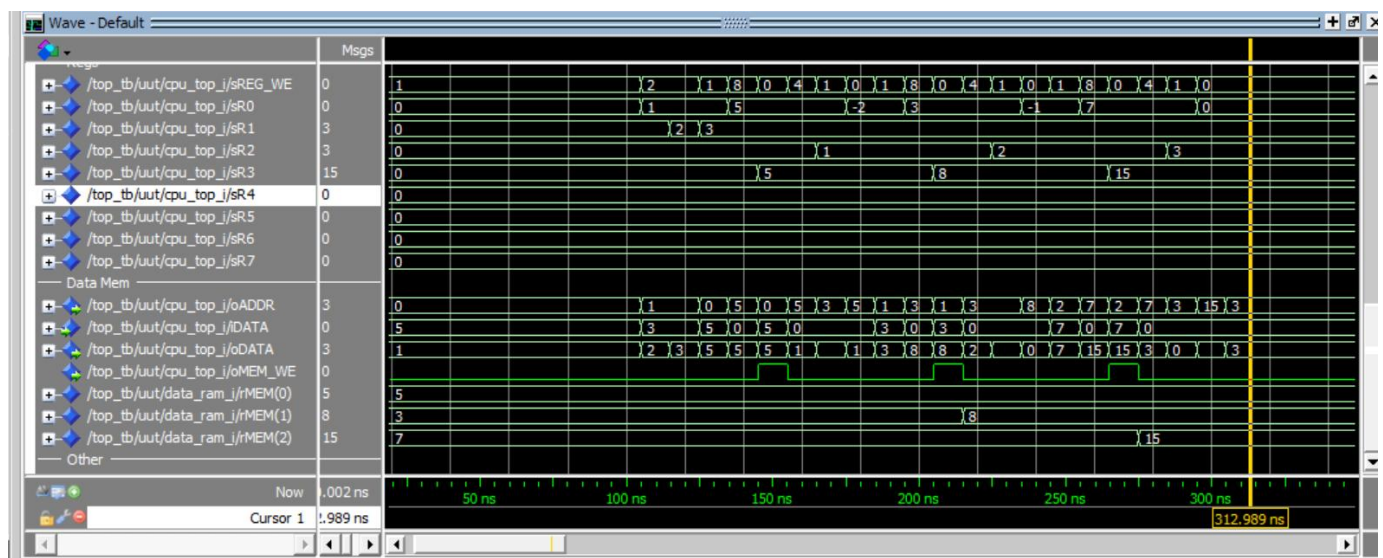
PITANJE: Zašto su ove dve datoteke izmenjene i koje podatke sadrže iz **cumsum.asm**?

S obzirom na to da su promenjene vhdL datoteke, neophodno je i **ponovo prevesti projekat** koji opisuje procesor.

NAPOMENA: Svaka promena u datoteci koja sadrži asemblerski program zahteva i ponovno asembliranje, a nakon toga i prevođenje celog projekta u Quartusu.

ZADATAK: Asemblirajte i pokrenite simulaciju za prvi primer **cumsum.asm**. Proverite ispravnost rezultata posmatranjem vrednosti registara i memorije. Nakon toga, izmenite primer menjanjem podataka nakon **.data** direktive i napišite isti program bez korišćenja petlje. Upišite krajnji rezultat u registar R7, kao i u memoriju za podatke na adresi 0x7.

Na Slici 3-2 je prikazan rezultat simulacije nakon izvršenog asemblerskog programa. U registru **R3** je suma svih učitanih vrednosti iz memorije za podatke ($5 + 3 + 7 = 15$), kao i u memoriji za podatke u signalu **rMEM(2)**.

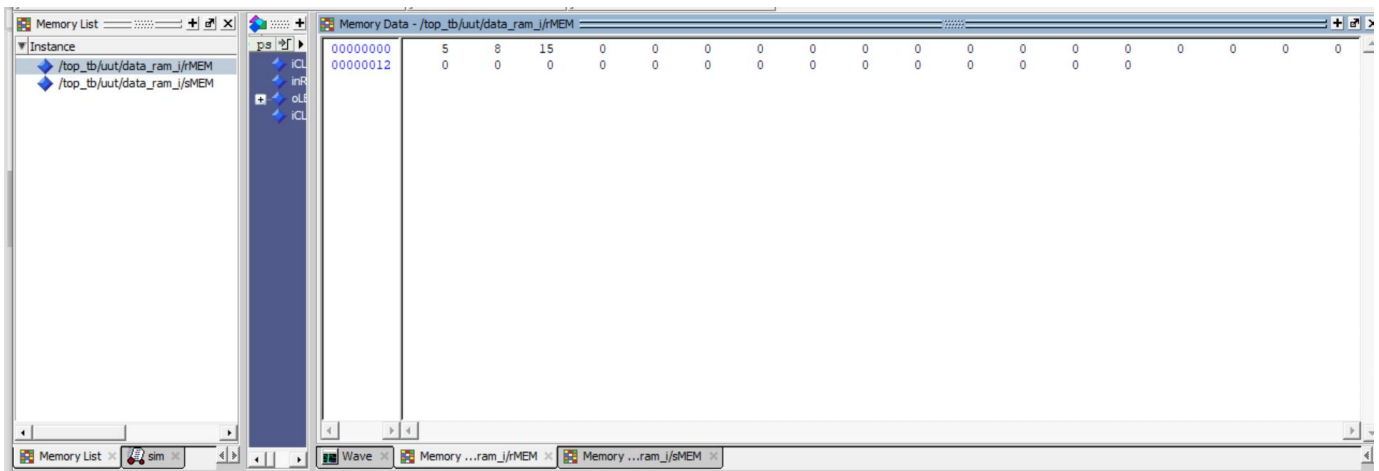


Slika 3-2 – Simulacija u ModelSim alatu

Ako želite da proverite samo sadržaj memorije, otvorite jezičak **Memory List** (u donjem levom uglu) i prikažite trenutni sadržaj memorije u signalu **rMEM**, kao na Slici 3-3.

VAŽNO: Ukoliko želite da asemblirate neki drugi asemblerski program, potrebno je da postavite datoteku u **src/asm** direktorijum i da u skripti **asm.tcl** pronađete liniju koda za postavljanje naziva datoteke i zatim je promenite:

```
set asm_file "src/asm/NOVI_ZADATAK.asm.txt"
```

Slika 3-3 – Prikaz sadržaja memorije – Memory List

4. Petlje u asemblerskom jeziku

Petlje se u svim programskim jezicima koriste onda kada želimo da izvršimo istu operaciju više puta. Naravno to važi za operacije koje je moguće izvršiti iterativno i u ovoj vežbi ćemo uraditi primere sa **for** i **while** petljom.

U nastavku je urađen zadatak koji u nizu od 10 elemenata broji koliko ima pozitivnih, negativnih i koliko je brojeva jednako **NULL**. Broj pozitivnih brojeva treba upisati u registar R7, broj negativnih u R6, a broj NULA u R5.

```
int N = 10;
short a[10] = {-3, 0, 0, 7, 12, -6, 15, -1, 0, 11};

for(i=0; i<N; i++)
{
    R1 = a[i];

    if(R1 == 0)
        R5++;
    elseif(R1 < 0)
        R6++;
    else
        R7++;
}
```

U nastavku sledi program napisan u LPRS1 asemblerskom jeziku (prikazan je samo deo koda koji sledi nakon .text direktive).

Svaki podatak koji treba menjati u petlji treba pažljivo čuvati u željenom registru i **paziti da se njegova vrednost ne prepíše**. Traženi rezultati treba da se nalaze redom u registrima R5, R6 i R7, a ostale registre koristimo proizvoljno za svaku od promenljivih koje koristimo u zadatku.

Poželjno je u komentaru napisati šta svaki od registara sadrži u kodu, kako bi kod bio čitljiviji i kako bi se brže i jednostavnije pronašle eventualne greške.


```
.data
10 // N = 10, adresa 0
-3 // a[0], adresa 1
0 // a[1], adresa 2
0 // a[2], adresa 3
7 // a[3], adresa 4
12 // a[4], adresa 5
-6 // a[5], adresa 6
15 // a[6], adresa 7
-1 // a[7], adresa 8
0 // a[8], adresa 9
11 // a[9], adresa 10

.text

/* R0 - i - adresa elementa iz memorije.
   R1 - učitani element iz memorije a[i]
   R3 - N = 10
   R5 - broj NULA
   R6 - broj negativnih
   R7 - broj pozitivnih
*/
begin:
    ld R3, R0           // R3 = N = 10, prva vrednost u memoriji je broj elemenata
    sub R0, R0, R0      // R0 = R0 - R0, R0 = 0
for:
    sub R2, R3, R0      // for(i = 0, i<10, i++)
    jmpz end            // ako je i > N , kraj
    inc R0, R0          // i++, vrednost uvećavamo pre LD jer je a[0] = broj elemenata
    ld R1, R0           // R1 = a[i]
    jmpz NULA           // if a[i] = 0 -> NULA++
    jmps NEG            // if a[i] = 0 -> NEG++
POZ:
    inc R7, R7          // else POZ++
    jmp for
NULA:
    inc R5, R5          // NEG++
    jmp for
NEG:
    inc R6, R6          // NULA++
    jmp for
end: jmp end // Beskonačna petlja.
```

ZADACI

1. Neparni brojevi

Vaš zadatak je da napišete program u asemblerskom jeziku koji u nizu od 10 elemenata pronalazi neparne brojeve. Ukupan broj neparnih brojeva treba smestiti u registar R7, kao i u memoriju za podatke, na lokaciju 0xF. U višem programskom jeziku, program bi izgledao ovako:

```
int N = 10;
int i = 0;

while (N != 0)
{
    R1 = a[i];
    i++;
    N--;
    if(R1(0) == 1)
        R7++;
}
MEM[0xF] = R7;
```

Dodatak: Emulator

U simulaciji je pored provere ispravnosti programa moguće i pronalaženje grešaka, koje se uglavnom svodi na proveru vrednosti registara u zadatom trenutku, tačnije u toku izvršavanja određene instrukcije.

PITANJE: Koji signal u simulaciji pokazuje koja se instrukcija izvršava u određenom trenutku?

Kako bismo olakšali pronalaženje grešaka i kontrolisano izvršavali programe napisane u asemblerskom jeziku, koristićemo **LPRSemu** softverski paket.

Softverski paket se sastoji iz dva programa: **lprsemu** i **lprsasm**. Prvi se koristi za emulaciju i verifikaciju ispravnosti rada asemblerskih programa, dok se drugi koristi za prevođenje u VHDL opis programske i radne memorije za referentnu implementaciju procesora.

Softverski alat se može preuzeti na dva načina: u binarnom izdanju na platformi GitHub, ili prevođenjem izvornog koda pomoću *Cargo* upravljača paketa za programski jezik *Rust*.

Primer prevođenja na operativnom sistemu *Ubuntu 22.04*:

```
curl https://sh.rustup.rs -sSf | sh
rustup install nightly-x86_64-unknown-linux-gnu
cargo install lprsemu
```

Adresa najnovijeg binarnog izdanja na GitHub-u:

<https://github.com/filiparag/ftn-lprsemu/releases/latest/>

Pokretanje emulatora za konkretni program se vrši pozivanjem **lprsemu** programa sa prosleđenom relativnom ili apsolutnom putanjom do datoteke koja sadrži asemblerski program:

```
lprsemu ./primer.asm
```

NAPOMENA: Emulator se takođe nalazi u okviru postavke za lab9, u folderu **bin**.

Prilikom pokretanja, moguće je dobiti grešku ukoliko programska datoteka nije dobro formatirana. Npr. ako je izostavljen treći operand instrukcije za sabiranje, dobija se sledeća greška:

```
Parsing error:  --> 12:53
|
12 |          add    R0, R0 # Сабирање чиниоца са тренутном сумом
|                               ^---
= expected register
```

Ukoliko je program sintaksno ispravan, emulator će se pokrenuti i dobiće se prikaz sličan sledećem:

```
Registers
| R0:      0 | R1:      0 | R2:      0 | R3:      0 |
| R4:      0 | R5:      0 | R6:      0 | R7:      0 |
Flags [ zero: false ] [ sign: false ] [ carry: false ]
Program counter: 0
Runtime counter: 0
Data memory
|  0 |      1
| ... |      0
Program memory
|    | kraj:
|  0 |      jmp    0 (kraj) <=
| ... | nop
lprsemu >>
```

Izlaz iz emulatora se vrši slanjem **SIGINT** signala, tj. prečicom **CTRL+C**.

U odeljku **“interaktivna konzola”** su prikazane vrednosti specijalnih i opštenamenskih registara. Podrazumevani zapis broja u registrima opšte namene je označen decimalni. U odeljku Interaktivna konzola je opisan postupak promene prikazanog zapisa.

U odeljku **Data memory** su prikazane sve reči u memoriji za podatke. Ako se na kraju nalaze uzastopne nule, one će biti prikazane skraćeno sa tri tačke. Podrazumevani zapis brojeva u memoriji je označen decimalni.

U odeljku **Program memory** prikazane su sve instrukcije u programskoj memoriji i propratne oznake. Ukoliko memorija nije popunjena do kraja, na dnu se nalaze tri tačke i specijalna instrukcija „nop“. Podrazumevano izvršavanje programa stoji na prvoj pojavi te instrukcije.

Na liniji asemblerskog koda koja će biti sledeća izvršena se nalazi simbol strelice „<=“.

Na linijama na kojima je aktivna tačka prekida se dodatno nalazi simbol zvezdice u zagradi: „(*)“.

Ukoliko pri izvršavanju asemblerskog koda dolazi do greške, recimo pokušaj učitavanja vrednosti u registar sa memorijske adrese van opsega, emulacija se zaustavlja i prikazuje grešku:

Emulation error: OutOfRange

Emulaciju je nemoguće nastaviti dalje bez resetovanja procesora, povratka unazad ili prinudnog skakanja na drugu instrukciju.

Interaktivna konzola

U nastavku su opisane komande navedene, sa skraćenim oblikom u zagradi:

- **print** (p) – Ispisuje trenutno stanje procesora.
- **load-file** (l) – Učitava program iz datoteke prosleđene kao prvi argument.
- **radix** (d) – Menja zapis prikazanih vrednosti u registrima i memoriji na osnovu prvog argumenta:
 - u: neoznačeni decimalni brojevi
 - s: označeni decimalni brojevi
 - x: neoznačeni heksadecimalni brojevi
 - b: neoznačeni binarni brojevi
- **run** (r) – Nastavlja izvršavanje programa do prve sledeće tačke prekida.
- **run all** (ra) – Nastavlja izvršavanje programa od trenutne do poslednje instrukcije.
- **step**(s) – Izvršava sledeću instrukciju. Podrazumevana komanda, izvršava se ako se samo pritisne Return.
- **undo** (u) – Resetuje procesor i izvršava sve instrukcije do prethodne.
- **breakpoint** (b) - Postavlja ili uklanja tačku prekida na instrukciji čija je adresa prvi parametar komande.
- **breakpoint-clear** (bc) – Uklanja sve tačke prekida.
- **jump** (j) – Skače na instrukciju čija je adresa prvi parametar komande.
- **reset** (x) – Resetuje stanje procesora na početno, bez uklanjanja tački prekida.
- **help** (h) – Ispisuje uputstvo za korišćenje interaktivne konzole.

ZAKLJUČAK

Upravo ste programirali LPRS1 procesor. U ovoj vežbi smo napisali i izvršili jednostavne programe napisane u asemblerskom jeziku. U daljem radu ćemo koristiti sve složenije programe. Sada bi trebali da razumete vezu između Asemblera kao prevodioca i programa napisanog na asemblerskom jeziku, kao i način izvršavanja generisanih instrukcija koje su rezultat prevođenja programa napisanog na assemblerkom jeziku.